

Master Notes — Recursion

Part 1c: Fundamentals, Backtracking, Divide & Conquer, Memoization, Advanced Techniques (C++, FAANG-condensed)

Contents

1	Recursion Fundamentals	2
2	Backtracking	5
3	Divide and Conquer	9
4	Recursion with Memoization (Top-Down DP Bridge)	13
5	Advanced Recursion Techniques	16

1 Recursion Fundamentals

One-line definition

A function that solves a problem by calling itself on a smaller version of the same problem, until it hits a case simple enough to answer directly (the base case).

Intuition

Think of Russian nesting dolls: to open the big doll, you open the one inside it, and the one inside that, until you reach the smallest doll that just opens with no doll inside — that smallest doll is the base case. Every recursive function needs exactly this: a point where it stops calling itself.

```
factorial(4)
= 4 * factorial(3)
  = 3 * factorial(2)
    = 2 * factorial(1)
      = 1 * factorial(0)
        = 1                <- base case, stops here
```

The Call Stack

Every function call — recursive or not — gets its own "frame" on the call stack holding its parameters and local variables. A recursive call doesn't share this frame with its caller; it gets a brand new one. So `factorial(4)` briefly has 5 frames alive at once (one for each call from 4 down to 0), and they pop off one by one as each call returns.

```
Frame for factorial(0) <- top of stack, returns first
Frame for factorial(1)
Frame for factorial(2)
Frame for factorial(3)
Frame for factorial(4) <- bottom of stack, called first
```

This is why deep recursion can crash your program: if the chain of calls goes too deep (say, recursing 200,000 times), you run out of stack memory — a "stack overflow." This is the single most common way recursion goes wrong on large inputs.

Operations

1. Basic Structure: Base Case + Recursive Case

```
int factorial(int n) {
    if (n == 0) return 1;           // base case
    return n * factorial(n - 1);    // recursive case
}
```

Every recursive function needs both parts. Skip the base case and it recurses forever (well, until the stack overflows). Skip making real progress toward the base case (e.g. calling `factorial(n)` instead of `factorial(n-1)`) and it never terminates either.

2. Linear Recursion (one recursive call per invocation)

Factorial above is linear recursion — one call leads to exactly one more call. $O(n)$ time, $O(n)$ space (one stack frame per call).

3. Binary / Multiple Recursion (more than one recursive call)

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);    // two recursive calls per
        invocation
}
```

Each call branches into two more calls, so the total number of calls grows like 2^n — this is **exponential time**, even though the problem itself feels simple. This is the single biggest trap in naive recursion, and it's exactly what memoization (Section 4) fixes.

4. Recursion vs. Iteration

Anything recursive can be rewritten as a loop, and vice versa. Recursion is usually easier to read for problems that are naturally defined in terms of smaller versions of themselves (trees, backtracking, divide-and-conquer); iteration is usually faster and safer from stack overflow for simple repeated work (like summing a list).

Patterns

Interviewer says	Think
"defined in terms of a smaller version of itself"	recursion
"can you do it without recursion?"	convert to a loop, or simulate the call stack manually
"why is this so slow for large n ?"	check for repeated (overlapping) recursive calls
"will this crash on large input?"	check recursion depth vs. stack size

Complexity Table

Pattern	Time	Space (stack)
Linear recursion (factorial)	$O(n)$	$O(n)$
Binary recursion, no memo (naive Fibonacci)	$O(2^n)$	$O(n)$
Binary recursion, with memo	$O(n)$	$O(n)$

Interview Follow-ups — Answered

- **"Can you rewrite this iteratively?"** Almost always yes — either restructure as a loop directly (simple linear recursion), or use an explicit stack/queue to simulate the call frames (needed for tree/graph recursion). See Section 5 for the general technique.
- **"What's the real space complexity?"** Don't forget the call stack — even if you use no extra data structures, recursion depth d costs $O(d)$ space. This is easy to forget when focused only on explicit variables.

- **”Why is naive Fibonacci exponential?”** Because `fib(n-2)` gets recomputed from scratch inside both `fib(n-1)` and directly — the same subproblems are solved over and over. Memoization (Section 4) caches each answer so it’s only computed once.

Common Bugs

- Missing or wrong base case $\Rightarrow$ infinite recursion $\Rightarrow$ stack overflow.
- Recursive call doesn’t shrink the problem (e.g. calling with the same `n`) $\Rightarrow$ never terminates.
- Forgetting that each call gets fresh local variables — mutating a shared/global variable across calls can produce surprising bugs if not intentional.
- Off-by-one in the base case (e.g. `n == 1` instead of `n == 0`) silently gives wrong answers.

Checklist

1. Does every recursive path eventually hit a base case?
2. Does each recursive call move strictly closer to the base case?
3. How deep can recursion go on the largest allowed input — is stack overflow a risk?
4. Are there overlapping/repeated subproblems? If yes, consider memoization.

Practice Problems

Difficulty	Problem	Source
Easy	Factorial / Sum of Digits	Standard warm-up
Easy	Fibonacci Number	LeetCode 509
Easy	Reverse a String Recursively	Standard warm-up
Easy	Power of Two (recursive check)	LeetCode 231
Medium	Pow(x, n)	LeetCode 50

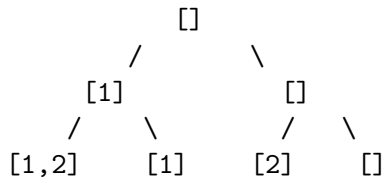
2 Backtracking

One-line definition

Recursion with a twist: try a choice, recurse to see if it leads anywhere good, and if it doesn't, *undo* the choice and try the next one. It's how you explore every possible option without missing any.

Intuition

Subsets of [1, 2]:



At every element, you make a choice (include it or don't), explore what happens, then step back and try the other choice. "Stepping back" — undoing whatever you just did before trying the next option — is the one thing that separates backtracking from plain recursion.

Operations

1. Subsets (Power Set)

Complexity: $O(2^n \cdot n)$ time (every subset is copied to the output), $O(n)$ recursion depth.

```
void subsets(vector<int>& nums, int idx, vector<int>& cur, vector<vector<
int>>& res) {
    if (idx == nums.size()) { res.push_back(cur); return; }
    // choice 1: skip nums[idx]
    subsets(nums, idx + 1, cur, res);
    // choice 2: include nums[idx]
    cur.push_back(nums[idx]);
    subsets(nums, idx + 1, cur, res);
    cur.pop_back(); // undo - this is the "backtrack"
                    step
}
```

2. Permutations

Complexity: $O(n! \cdot n)$ time — there are $n!$ permutations, each costing $O(n)$ to build/copy.

```

void permute(vector<int>& nums, vector<bool>& used, vector<int>& cur,
            vector<vector<int>>& res) {
    if (cur.size() == nums.size()) { res.push_back(cur); return; }
    for (int i = 0; i < nums.size(); i++) {
        if (used[i]) continue;
        used[i] = true;
        cur.push_back(nums[i]);
        permute(nums, used, cur, res);
        cur.pop_back();           // undo
        used[i] = false;         // undo
    }
}

```

3. Combinations / Combination Sum

Idea: same choose-or-skip pattern as subsets, but stop early once the target is exceeded (a form of pruning — cutting off a branch before fully exploring it, because we already know it can't work).

```

void combinationSum(vector<int>& cand, int target, int idx, vector<int>&
                  cur, vector<vector<int>>& res) {
    if (target == 0) { res.push_back(cur); return; }
    if (target < 0 || idx == cand.size()) return; // pruned: dead end
    cur.push_back(cand[idx]);
    combinationSum(cand, target - cand[idx], idx, cur, res); // reuse
    // same element
    cur.pop_back();
    combinationSum(cand, target, idx + 1, cur, res); // move on
    // without it
}

```

4. N-Queens (constraint-heavy backtracking)

Idea: place one queen per row; before placing in a column, check it doesn't conflict with any queen already placed (same column, or same diagonal). If no valid column exists in a row, backtrack to the previous row and try a different column there. $O(n!)$ time worst case, but pruning invalid placements early makes it fast in practice for the usual interview sizes ($n \leq 9$ or so).

5. Sudoku Solver

Same shape as N-Queens: try a digit in an empty cell, recurse, and if it leads to a dead end, undo and try the next digit. Checking row/column/box validity before recursing (instead of after) is what keeps this fast enough to run in practice.

6. Word Search on a Grid

DFS from every starting cell, marking visited cells so you don't reuse them, and un-marking on the way back out (the backtrack step) so other starting points can reuse those cells.

Handling Duplicates

When the input has duplicate values (e.g. `[1,2,2]`) and you don't want duplicate subsets/permutations in the output: sort the array first, then at each recursion level skip over a value if it's the same as the one you just tried at that same level — this avoids generating the same combination twice.

Patterns

Interviewer says	Think
"generate all subsets/combinations/permutations"	backtracking, choose-or-skip / swap pattern
"place items under constraints (queens, sudoku)"	backtracking with early constraint checking
"find all paths / all ways to reach a target"	backtracking with pruning
"avoid duplicate results"	sort input, skip repeated values at the same recursion level

Complexity Table

Problem	Time	Space (stack)
Subsets	$O(2^n \cdot n)$	$O(n)$
Permutations	$O(n! \cdot n)$	$O(n)$
Combination Sum	problem-dependent, exponential worst case	$O(target)$
N-Queens	$O(n!)$ worst case, much less with pruning	$O(n)$

Interview Follow-ups — Answered

- **"How is backtracking different from brute force?"** Brute force generates everything and checks it after; backtracking checks constraints *as it builds*, so it can abandon a bad path early instead of wasting time completing it — this pruning is what makes backtracking practical.
- **"How do you avoid duplicate results?"** Sort the input, then skip a choice if it's equal to the previous choice tried at the same recursion depth (the previous one already covered every outcome that starts with that value).
- **"Can you do this iteratively?"** Usually possible with an explicit stack holding partial state, but it's rarely worth the added complexity — recursion is the natural fit here.

Common Bugs

- Forgetting the "undo" step (`pop_back()`, resetting a `used` flag) — leaves stale state that corrupts later branches.
- Passing containers by value instead of by reference, causing the "undo" step to be pointless (each call has its own copy) or missing entirely.
- Not pruning early enough — checking constraints only after building a complete (invalid) answer instead of during construction.
- Wrong duplicate-skipping condition — skipping too early can also remove valid answers.

Checklist

1. What's the choice being made at each recursive step?
2. Is there a way to detect a dead end early and prune, instead of exploring the full branch?
3. Does every explored branch get properly undone before the next one starts?
4. Do duplicates in the input need special handling?

Practice Problems

Difficulty	Problem	Source
Medium	Subsets	LeetCode 78
Medium	Subsets II (with duplicates)	LeetCode 90
Medium	Permutations	LeetCode 46
Medium	Combination Sum	LeetCode 39
Medium	Palindrome Partitioning	LeetCode 131
Hard	N-Queens	LeetCode 51
Hard	Sudoku Solver	LeetCode 37
Hard	Word Search II	LeetCode 212

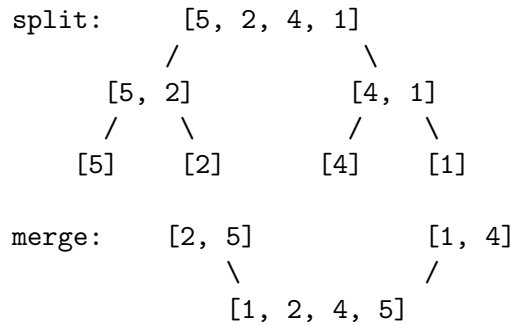
3 Divide and Conquer

One-line definition

Break a problem into smaller, independent subproblems of the same kind, solve each one recursively, then combine their answers into the final result.

Intuition

Merge Sort on [5, 2, 4, 1]:



The "split" half does almost no work — it's the "merge" (combine) step where the real work of putting things back together happens. This split-then-combine shape is what defines divide and conquer, and it's different from backtracking: the subproblems here are fully independent, so there's nothing to "undo."

Operations

1. Merge Sort

Complexity: $O(n \log n)$ time (always — no bad input can make it worse), $O(n)$ extra space for the merge step. **Stable** (equal elements keep their relative order).

```

void merge(vector<int>& a, int l, int m, int r) {
    vector<int> tmp;
    int i = l, j = m + 1;
    while (i <= m && j <= r) tmp.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
    ;
    while (i <= m) tmp.push_back(a[i++]);
    while (j <= r) tmp.push_back(a[j++]);
    for (int k = 0; k < tmp.size(); k++) a[l + k] = tmp[k];
}
void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}

```

2. Quick Sort

Complexity: $O(n \log n)$ average, $O(n^2)$ worst case (already-sorted input with a naive pivot choice), $O(\log n)$ extra space (in-place, just recursion stack). Faster in practice than merge sort due to better cache behavior, but loses the worst-case guarantee and stability.

```

int partition(vector<int>& a, int l, int r) {
    int pivot = a[r], i = l;
    for (int j = l; j < r; j++)
        if (a[j] < pivot) swap(a[i++], a[j]);
    swap(a[i], a[r]);
    return i;
}
void quickSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int p = partition(a, l, r);
    quickSort(a, l, p - 1);
    quickSort(a, p + 1, r);
}

```

Fixing the worst case: pick a random pivot (or median-of-three) instead of always the last element — makes the $O(n^2)$ worst case extremely unlikely on any real input.

3. Quickselect (Kth Largest / Smallest)

Idea: same partitioning as quicksort, but only recurse into the *one* side that contains the target index — throw away the other side entirely instead of sorting it. $O(n)$ average time (vs. $O(n \log n)$ to fully sort), $O(n^2)$ worst case with a bad pivot.

4. Binary Search (the simplest divide and conquer)

Complexity: $O(\log n)$ time, $O(1)$ space iteratively (or $O(\log n)$ stack recursively). Every step throws away half the remaining search space.

```

int binarySearch(vector<int>& a, int target) {
    int lo = 0, hi = a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        else if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

```

Variants: search in a rotated sorted array, find the first/last occurrence of a target, find a peak element — all reuse this same "shrink the search space by half" idea with a different condition for which half to keep.

5. Maximum Subarray (Divide and Conquer version)

Idea: split the array in half; the best subarray is either entirely in the left half, entirely in the right half, or crosses the midpoint (compute that case by expanding outward from the middle). $O(n \log n)$ time — Kadane's algorithm (a simple linear scan, covered in DP/array notes) solves the same problem in $O(n)$, so this version is mainly useful as a "can you think of another way?" follow-up.

6. Closest Pair of Points (brief overview)

Sort points by x-coordinate, split in half, recursively find the closest pair in each half, then check only the narrow strip of points near the dividing line for a closer cross-boundary pair. $O(n \log n)$ total — a classic example of divide and conquer applied to computational geometry.

How to Estimate Time Complexity Without Memorizing a Formula

Most divide-and-conquer recurrences fall into one of three familiar shapes — you don't need the Master Theorem formula to recognize them in an interview:

- **Split in half, linear work to combine** (merge sort): $O(n \log n)$.
- **Split in half, constant work to combine** (binary search): $O(\log n)$.
- **Branch into multiple calls without shrinking much** (naive recursive Fibonacci): exponential, $O(2^n)$ or similar.

If you can say which of these three shapes your recursion tree looks like, you can state the complexity confidently without doing algebra live in an interview.

Patterns

Interviewer says	Think
"sort this, with a time guarantee regardless of input"	merge sort ($O(n \log n)$ always)
"sort in-place, fastest in practice"	quick sort (mention worst case + random pivot fix)
"find the kth largest/smallest, without fully sorting"	quickselect
"search a sorted (or rotated-sorted) array"	binary search
"split the problem in half and combine"	divide and conquer, generally

Complexity Table

Algorithm	Average Time	Worst Time	Space
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Quickselect	$O(n)$	$O(n^2)$	$O(1)$ extra
Binary Search	$O(\log n)$	$O(\log n)$	$O(1)$ iterative

Interview Follow-ups — Answered

- **”Why does quicksort have a bad worst case but merge sort doesn’t?”** Quicksort’s split depends on the pivot — a consistently bad pivot (e.g. always picking the smallest/largest element) gives lopsided splits and $O(n^2)$. Merge sort always splits exactly in half regardless of the data, so it’s always $O(n \log n)$.
- **”How do you fix quicksort’s worst case?”** Pick the pivot randomly (or use median-of-three) instead of a fixed position — makes worst-case input astronomically unlikely in practice.
- **”When would divide and conquer not help?”** When subproblems aren’t independent (they overlap) — in that case you want memoization/DP instead, since D&C would redo the same work repeatedly.

Common Bugs

- Off-by-one in the midpoint or partition index causing infinite recursion or skipped elements.
- Forgetting the base case $l \geq r$ in sort functions.
- Quickselect: forgetting to recurse into only one side (recursing into both defeats the point).
- Integer overflow computing $mid = (lo + hi) / 2$ on huge arrays — prefer $lo + (hi - lo) / 2$.

Checklist

1. Are the subproblems truly independent, or do they overlap (which would call for memoization instead)?
2. Does the combine step dominate the cost, or is it cheap?
3. Worst-case guarantee needed? (favors merge sort over quicksort)
4. In-place / memory-constrained? (favors quicksort over merge sort)

Practice Problems

Difficulty	Problem	Source
Easy	Binary Search	LeetCode 704
Medium	Search in Rotated Sorted Array	LeetCode 33
Medium	Kth Largest Element in an Array	LeetCode 215
Medium	Maximum Subarray	LeetCode 53
Medium	Sort an Array	LeetCode 912
Hard	Median of Two Sorted Arrays	LeetCode 4
Hard	Count of Smaller Numbers After Self	LeetCode 315

4 Recursion with Memoization (Top-Down DP Bridge)

One-line definition

Plain recursion, plus a cache that remembers the answer to each subproblem the first time it's computed — so if the same subproblem comes up again, you just look it up instead of recomputing it.

Intuition

fib(5) without memo: <pre> fib(5) / \ fib(4) fib(3) / \ / \ fib(3) fib(2) (recomputed!)</pre>	fib(5) with memo: <pre> fib(5) / \ fib(4) fib(3) <- fib(3) looked up, not recomputed / \ fib(3) fib(2) (cached the first time)</pre>
--	--

Notice `fib(3)` gets computed from scratch twice on the left. Once you've computed it the first time, there's no reason to ever do that work again — just remember the answer. That's the entire idea.

Operations

1. Memoized Fibonacci

Complexity: $O(n)$ time (each value from 0 to n is computed exactly once), $O(n)$ space for the cache plus $O(n)$ recursion stack.

```
unordered_map<int, long long> memo;
long long fib(int n) {
    if (n <= 1) return n;
    if (memo.count(n)) return memo[n];           // already solved - reuse
    it
    return memo[n] = fib(n - 1) + fib(n - 2);    // solve once, then cache
}
```

The only change from naive recursion is two lines: check the cache before recursing, and store the result after computing it. This turns $O(2^n)$ into $O(n)$.

2. Identifying the Memo Key

The cache key is whatever combination of parameters uniquely identifies a subproblem. For Fibonacci it's just `n`. For a grid path-counting problem it might be `(row, col)`. For a string problem it's often `(i, j)` — a pair of indices into the string. **Rule of thumb:** if calling the function twice with the exact same arguments always gives the exact same answer, those arguments are your key.

3. Example: Counting Grid Paths

Problem: count paths from the top-left to bottom-right of a grid, moving only right or down.
Idea: `paths(r, c) = paths(r+1, c) + paths(r, c-1)`, memoized on `(r, c)`. $O(\text{rows} \times \text{cols})$ time and space instead of exponential.

Memoization (Top-Down) vs. Tabulation (Bottom-Up)

Both cache answers to subproblems — the difference is direction. **Memoization** starts from the original question and recurses downward, caching as it goes (you only compute the subproblems you actually need). **Tabulation** starts from the smallest subproblems and builds upward in a loop, filling a table (no recursion, no stack risk, but you compute every subproblem even if some aren't needed). **Rule of thumb:** start with memoization since it's usually easier to write correctly from the plain recursive version; switch to tabulation afterward if recursion depth becomes a problem.

Patterns

Interviewer says	Think
"count the number of ways to..."	memoized recursion
"find the minimum/maximum cost to..."	memoized recursion
"this naive recursion is too slow"	check for overlapping subproblems, add memoization
"can you optimize the space?"	consider switching to bottom-up tabulation

Complexity Table

Version	Time	Space
Naive recursion (Fibonacci)	$O(2^n)$	$O(n)$ stack
Memoized recursion	$O(n)$	$O(n)$ cache + $O(n)$ stack
Bottom-up tabulation	$O(n)$	$O(n)$, often reducible to $O(1)$

Interview Follow-ups — Answered

- **"How do you know memoization will help?"** Check whether the same (parameters) combination gets recomputed multiple times in the naive recursion tree — if yes, that's an overlapping subproblem, and memoization removes the repeated work.
- **"What if there are multiple parameters?"** Use a combined key — a pair/tuple, or a 2D array indexed by both parameters if the ranges are small and known in advance (faster than a hashmap).
- **"Does memoization fix the stack overflow risk?"** No — it fixes redundant *work*, not recursion *depth*. If the recursion still goes very deep, you still risk a stack overflow; switching to bottom-up tabulation removes recursion entirely if that's a concern.

Common Bugs

- Checking the cache *after* doing the recursive work instead of before — defeats the purpose.
- Using a memo key that doesn't fully capture the subproblem (missing a parameter that actually affects the answer) — causes wrong cached results to be reused.
- Forgetting to actually store the computed value into the cache before returning.
- Assuming memoization also reduces recursion depth — it doesn't (see follow-up above).

Checklist

1. What are the parameters that fully define a subproblem? That's your memo key.

2. Do I check the cache before recomputing, and store the result after computing?
3. Is recursion depth itself a concern, independent of redundant work?
4. Would bottom-up tabulation be simpler or safer here?

Practice Problems

Difficulty	Problem	Source
Easy	Climbing Stairs	LeetCode 70
Easy	Fibonacci Number	LeetCode 509
Medium	Coin Change	LeetCode 322
Medium	Unique Paths	LeetCode 62
Medium	Longest Common Subsequence	LeetCode 1143
Medium	House Robber	LeetCode 198
Hard	Word Break II	LeetCode 140

5 Advanced Recursion Techniques

One-line definition

The extra tools and tricks for when plain recursion isn't quite enough — making recursion safer (no stack overflow), converting it to loops when needed, and reasoning clearly about problems that use more than one recursive function.

1. Tail Recursion

Definition: a recursive call is "in tail position" if it's the very last thing the function does — nothing happens after it returns.

```
// NOT tail recursive: multiplication happens AFTER the recursive call
// returns
int factorial(int n) { return n == 0 ? 1 : n * factorial(n - 1); }

// Tail recursive: the recursive call is the last action, nothing left to
// do after
int factorialTail(int n, int acc = 1) {
    if (n == 0) return acc;
    return factorialTail(n - 1, n * acc);    // pass the running result
                                             forward
}
```

Important for C++ specifically: some languages automatically turn tail recursion into a loop behind the scenes (no extra stack frames), but **C++ does not guarantee this** — it depends on the compiler and optimization settings. Don't rely on tail recursion alone to avoid stack overflow in C++; if depth is a real concern, convert to an explicit loop instead.

2. Converting Recursion to Iteration (Explicit Stack)

Idea: replace the implicit call stack with your own `std::stack`, storing whatever state each "call" would have needed. This is the general technique behind the iterative tree traversals covered in the Trees notes.

```
// Recursive countdown
void countdown(int n) { if (n == 0) return; cout << n << "\n"; countdown(n - 1); }

// Same thing, iterative, using an explicit stack instead of the call
// stack
void countdownIter(int n) {
    stack<int> st;
    st.push(n);
    while (!st.empty()) {
        int cur = st.top(); st.pop();
        if (cur == 0) continue;
        cout << cur << "\n";
        st.push(cur - 1);
    }
}
```


This trades stack-overflow risk (bounded by system stack size) for heap-allocated storage (bounded by available memory, usually much larger) — the standard fix when a recursive solution is correct but might recurse too deep on large inputs.

3. Mutual (Indirect) Recursion

Two or more functions call each other instead of a function calling itself directly.

```
bool isEven(int n) { return n == 0 ? true : isOdd(n - 1); }
bool isOdd(int n)  { return n == 0 ? false : isEven(n - 1); }
```

Same rules apply as regular recursion — every path still needs to reach a base case eventually. This pattern shows up naturally in recursive-descent parsers (e.g. an expression parser calling a term parser, which calls a factor parser, which calls back into the expression parser for parenthesized expressions).

4. Reasoning About Complexity from the Recursion Tree

For any recursive function, draw (or imagine) the tree of calls: how many calls happen at each “level,” and how much work each call does on its own (excluding its recursive calls). Multiplying those together and summing across levels gives the total time — this informal approach covers the vast majority of interview recursion problems without needing to memorize a formula (see the three shapes listed in Section 3).

5. Fast Exponentiation (Recursive Divide and Conquer on the Exponent)

Idea: to compute x^n , don’t multiply x by itself n times — instead halve the exponent each time: $x^n = (x^{n/2})^2$ (and handle odd n with one extra multiplication). $O(\log n)$ time instead of $O(n)$ — a small but classic example of applying the divide-and-conquer mindset to something that doesn’t look like a “data structure” problem at first glance.

```
double fastPow(double x, long long n) {
    if (n == 0) return 1.0;
    if (n < 0) return 1.0 / fastPow(x, -n);
    double half = fastPow(x, n / 2);
    return (n % 2 == 0) ? half * half : half * half * x;
}
```

Patterns

Interviewer says	Think
”can this overflow the stack on large input?”	convert to iterative with an explicit stack
”is this tail recursive? does it matter here?”	yes conceptually, but C++ won’t auto-optimize it
”two recursive things depend on each other”	mutual recursion
”compute x^n efficiently”	fast exponentiation, halve the exponent each step

Interview Follow-ups — Answered

- **”Does C++ optimize tail recursion automatically?”** Not reliably — unlike languages such as Scheme, C++ has no language guarantee of tail-call optimization. Some compilers do it under optimization flags, but you should never depend on it for correctness on deep recursion.
- **”How would you handle recursion depth in the millions?”** Convert to an iterative version using an explicit `stack` (or restructure as a loop if the recursion is linear) — heap memory for an explicit stack is typically far larger than the call stack’s default size.
- **”What’s the difference between backtracking, divide and conquer, and memoized recursion?”** Backtracking explores choices and undoes them when they don’t work out; divide and conquer splits into independent subproblems with no undoing needed; memoized recursion is about avoiding repeated work on *overlapping* subproblems. All three are recursion, but solving different kinds of problems.

Common Bugs

- Assuming tail recursion is automatically stack-safe in C++ — it usually isn’t.
- When converting to an explicit stack, forgetting to push state in the reverse order it should be processed (stacks are LIFO — last pushed is popped first).
- Mutual recursion missing a shared base case, causing infinite back-and-forth calls.
- Fast exponentiation: forgetting the negative-exponent case, or an off-by-one in the odd/even split.

Checklist

1. Is recursion depth actually bounded safely for the largest expected input?
2. If converting to iteration, what state does each ”frame” need, and does my explicit stack hold all of it?
3. Am I about to write an $O(n)$ recursive solution to something with an $O(\log n)$ divide-and-conquer version (like exponentiation)?

Practice Problems

Difficulty	Problem	Source
Medium	Pow(x, n)	LeetCode 50
Medium	Convert BST Inorder to Iterative	LeetCode 94 (iterative variant)
Medium	Flatten Nested List Iterator	LeetCode 341
Hard	Design a recursive-descent expression parser	Interview staple
Hard	Convert deep recursive DFS to iterative (explicit stack)	Interview staple